

Relatório de Testes de Integração e API — Backend nassauTickets

Validação funcional, regras de negócio e segurança da API REST

Item	Especificação
Projeto	nassauTickets
Escopo	API REST, Regras de Negócio de Atendimento, Filas de Prioridade e Segurança
Ferramenta de Execução	Thunder Client (VS Code)
Ambiente de Teste	Node.js v20+, Express, MySQL 8 / MariaDB (Sequelize ORM)
Coleção de Testes	thunder-collection_nassautickets.json
Status Geral	Aprovado (100% de conformidade)

1. Resumo Executivo

Este documento consolida a validação de todos os requisitos funcionais, regras de negócio e restrições de segurança do backend do sistema nassauTickets, executados por meio do Thunder Client.

Critérios Validados

- **Padronização de Numeração:** formato diário YYMMDD-PPSQ com prefixo temporal e sequência diária por categoria.
- **Algoritmo de Prioridade:** alternância rigorosa entre o grupo prioritário (SP) e o grupo não prioritário (SE / SG), garantindo escoamento sem inanição (*starvation*).
- **Comportamento em Fila Vazia:** retorno estruturado e limpo, sem disparar falhas internas.
- **Ciclo de Vida da Senha:** transição auditável entre os estados canônicos, com *timestamps* no banco de dados.
- **Impacto nos Relatórios Gerenciais:** reflexo em tempo real do atendimento no consolidado diário.
- **Controle de Ausência:** validação do teto de duas chamadas e registro de não comparecimento.
- **Limite do Painel:** exibição restrita às 5 últimas senhas chamadas, em ordem cronológica reversa.
- **Segurança e RBAC:** autenticação JWT, validação de payloads vazios (400), credenciais incorretas (401), ausência de token (401) e segregação de perfil Atendente vs Gestor (403).
- **Resiliência a Erros de Cliente:** respostas consistentes na faixa 4xx, sem vazamento de exceções 500.

2. Pré-condições e Preparação do Ambiente

Antes da execução da bateria de testes, o ambiente foi inicializado com a base padrão:

```
# 1. Navegar até o diretório do backend
cd backend

# 2. Executar o seed de banco (cria guichês 1, 2, 3 e usuários de teste)
node seed.js

# 3. Inicializar o servidor da aplicação
npm start
```

Credenciais Canônicas para Testes

- **Atendente:** atendente1 / atendente123 (perfil: atendente)
- **Gestor:** gestor1 / gestor123 (perfil: gestor)

3. Matriz de Resultados dos Casos de Teste

ID	Cenário	Método e Endpoint	Payload / Condição	Status Esperado	Status Obtido	Resultado
TC-01	Login Atendente com sucesso	POST /auth/login	{"login":"atendente1","senha":"atendente123"}	200 OK	200 OK	Aprovado
TC-02	Login Gestor com sucesso	POST /auth/login	{"login":"gestor1","senha":"gestor123"}	200 OK	200 OK	Aprovado
TC-03	Login com senha incorreta	POST /auth/login	{"login":"atendente1","senha":"errada"}	401 Unauthorized	401 Unauthorized	Aprovado
TC-04	Login com corpo vazio	POST /auth/login	{}	400 Bad Request	400 Bad Request	Aprovado
TC-05	Chamada sem token JWT	POST /atendimento/chamar	Sem header Authorization	401 Unauthorized	401 Unauthorized	Aprovado
TC-06	Relatório sem token JWT	GET /relatorios/diario	Sem header Authorization	401 Unauthorized	401 Unauthorized	Aprovado
TC-07	Chamada com fila vazia	POST /atendimento/chamar	Guichê 1 (token Atendente, fila zerada)	200 OK	200 OK	Aprovado
TC-08	Emissão e formato da numeração	POST /senhas	Sequência: SP, SP, SE, SE, SG, SG, SP	201 Created	201 Created	Aprovado
TC-09	Emissão com tipo inválido	POST /senhas	{"tipo":"INVALIDO"}	400 Bad Request	400 Bad Request	Aprovado
TC-10	Ordem de prioridade (alternância)	POST /atendimento/chamar	7 chamadas consecutivas no guichê 1	200 OK	200 OK	Aprovado
TC-11	Painel: limite de 5 senhas	GET /senhas/painel	Consulta após 7 chamadas realizadas	200 OK	200 OK	Aprovado
TC-12	Iniciar atendimento	POST /atendimento/iniciar	{"senhaId":1,"atendenteId":1}	200 OK	200 OK	Aprovado
TC-13	Finalizar atendimento	POST /atendimento/finalizar	{"senhaId":1}	200 OK	200 OK	Aprovado
TC-14	Contabilização no Relatório Diário	GET /relatorios/diario	Token Gestor	200 OK	200 OK	Aprovado
TC-15	Atendente tentando ver relatório	GET /relatorios/diario	Token Atendente	403 Forbidden	403 Forbidden	Aprovado
TC-16	Rechamada (2ª chamada)	POST /atendimento/chamar-novamente	{"senhaId":2}	200 OK	200 OK	Aprovado
TC-17	3ª chamada inválida	POST /atendimento/chamar-novamente	{"senhaId":2} (já rechamada)	400 Bad Request	400 Bad Request	Aprovado
TC-18	Marcar não com parecimento	POST /atendimento/nao-compareceu	{"senhaId":2}	200 OK	200 OK	Aprovado
TC-19	Iniciar senha inexistente	POST /atendimento/iniciar	{"senhaId":99999,"atendenteId":1}	400 Bad Request	400 Bad Request	Aprovado
TC-20	Chamada com token malformado	POST /atendimento/chamar	Bearer token_invalido	401 Unauthorized	401 Unauthorized	Aprovado

4. Detalhamento dos Cenários Críticos

4.1 Validação do Formato de Numeração (RN03 / RF03)

Ao emitir as senhas sequenciais, o backend gera a chave única no formato YYMMDD-PPSQ, em que PP é o tipo da senha e SQ é o sequencial diário com três dígitos:

- 261001-SP001 (tipo: SP, sequencial diário: 001)
- 261001-SP002 (tipo: SP, sequencial diário: 002)
- 261001-SE001 (tipo: SE, sequencial diário: 001)
- 261001-SE002 (tipo: SE, sequencial diário: 002)
- 261001-SG001 (tipo: SG, sequencial diário: 001)
- 261001-SG002 (tipo: SG, sequencial diário: 002)
- 261001-SP003 (tipo: SP, sequencial diário: 003)

O prefixo de data garante unicidade e reinício automático da sequência diária, sem colisões entre dias diferentes.

4.2 Algoritmo de Prioridade e Alternância (RN01, RN02)

Com a fila contendo exatamente as senhas emitidas acima, foram efetuadas 7 requisições sucessivas a POST /atendimento/chamar. A ordem de entrega do algoritmo foi:

- 1 SP001 — inicia pelo grupo prioritário SP.
- 2 SE001 — alterna para o grupo SE|SG, priorizando SE sobre SG.
- 3 SP002 — retorna para SP.
- 4 SE002 — alterna para SE|SG, consumindo a próxima SE.
- 5 SP003 — retorna para SP.
- 6 SG001 — alterna para SE|SG; como a fila de SE esgotou, consome SG.
- 7 SG002 — filas de SP e SE esgotadas, consome o restante de SG.

Conclusão: o resultado obtido (SP001 → SE001 → SP002 → SE002 → SP003 → SG001 → SG002) atende rigorosamente à especificação de ordenação.

4.3 Auditoria dos 7 Estados Canônicos da Máquina de Estados

O ciclo de vida das entidades foi monitorado no banco de dados (tabela senhas):



A seta tracejada indica que a transição de não comparecimento também pode partir diretamente do estado CHAMADA.

- **EMITIDA**: registrado em `Senha.create()` como estado transiente inicial.
- **AGUARDANDO**: atribuído no retorno da rota de emissão (`POST /senhas`).
- **CHAMADA**: gravado em `POST /atendimento/chamar` (com preenchimento de `guicheId` e `primeiraChamada`).
- **CHAMADA_NOVAMENTE**: gravado em `POST /atendimento/chamar-novamente` (com `segundaChamada`).
- **EM_ATENDIMENTO**: gravado em `POST /atendimento/iniciar` (com `atendenteId` e `inicioAtendimento`).
- **ATENDIDA**: gravado em `POST /atendimento/finalizar` (com `fimAtendimento`).
- **NAO_COMPARECEU**: gravado em `POST /atendimento/nao-compareceu`.

4.4 Limitação do Painel (RN07 / RF08)

Após a chamada de 7 senhas, a requisição `GET /senhas/painel` retornou um vetor com exatamente 5 elementos, ordenados de forma decrescente por `primeiraChamada`:

SG002, SG001, SP003, SE002, SP002

As senhas SP001 e SE001 foram automaticamente descartadas da visão do painel, cumprindo o limite máximo de 5 exibições.

4.5 Controle de Acesso e Perfil (RBAC)

- **Atendente** acessando `GET /relatorios/diario`: bloqueado com HTTP 403 Forbidden (`{"erro": "Acesso restrito ao gestor"}`).
- **Gestor** acessando `GET /relatorios/diario`: aprovado com HTTP 200 OK, com contagem imediata da senha finalizada no campo `atendidas`: 1.

4.6 Tratamento de Erros de Cliente (4xx)

Todas as operações anômalas solicitadas pelo cliente (corpo vazio, tipos inexistentes, transições de estado proibidas, IDs inexistentes e tokens inválidos) resultaram em status 400, 401 ou 403. Nenhuma requisição gerou falha não tratada 500.

5. Instruções de Reprodução pelo Thunder Client

Importar Coleção e Ambiente no VS Code

- 1 Abra a aba do Thunder Client no VS Code.
- 2 Em **Collections**, clique no menu de opções → **Import** → selecione `thunder-collection_nassautickets.json`.
- 3 Em **Env**, clique em **Import** → selecione `thunder-environment_nassautickets.json`.

Executar Login

- 1 Execute a requisição **01.1 - Login Atendente**. Copie o token retornado e cole na variável `tokenAtendente` do ambiente.
- 2 Execute **01.2 - Login Gestor**. Copie o token e cole na variável `tokenGestor`.

Executar a Bateria Completa

- 1 Clique com o botão direito na coleção importada e escolha **Run Collection**.
- 2 Todas as 20 asserções automatizadas serão executadas em sequência com status **Passed**.